



Carrera:
Programación y WebMaster

Docente:
Mario Alonso Segovia Gutiérrez

Materia:
Proyecto de una Plataforma Virtual

Alumno(s):
Castro Blanco Manuel David
Maximiliano Dzul Pérez
Gerardo Alexander Diaz León
Pablo Leonel Salomón Campos

Fecha:
20 de Julio 2026

Tabla de contenido

Introducción.....	3
Descripción del Proyecto	3
Nombre del proyecto.....	3
Objetivo general.....	3
Problema que resuelve.....	3
Tecnologías utilizadas	3
Arquitectura general del sistema	4
Módulos principales	4
Estrategia General de Pruebas.....	6
Tipos de pruebas	6
Momento de ejecución	6
Responsables	7
Herramientas para documentar y registrar resultados.....	7
Justificación	8
Plan de Pruebas Unitarias	8
PU-01: Cálculo de fecha de egreso prevista.....	8
PU-02: Generación automática de folio de expediente.....	8
PU-03: Verificación de permisos por rol (RBAC)	9
PU-04: Cálculo de stock disponible de medicamento por lotes.....	9
PU-05: Validación de límite de permanencia (6 meses máximo)	10
PU-06: Validación de administración de medicamento con stock insuficiente	11
PU-07: Búsqueda difusa de personas por trigram (prevención de duplicados).....	11
Plan de Pruebas de Integración.....	12
Plan de Pruebas End-to-End (E2E).....	17
E2E-01: Admisión completa de un nuevo residente.....	17
E2E-02: Proceso clínico interdisciplinario y evaluación trimestral	18
E2E-03: Egreso de residente con cierre de expediente	19
E2E-04: Registro de bitácora diaria con servicios e incidencia	20
Matriz de Cobertura de Pruebas.....	21
Criterios de Aceptación	24
Funcionamiento correcto	24
Cumplimiento del requerimiento	24
Manejo adecuado de errores	24
Validación de datos.....	24

Integración correcta con otros módulos	24
Ausencia de errores críticos	25
Control de acceso.....	25
<i>Riesgos y Limitaciones</i>	25
<i>Conclusión.....</i>	30

Introducción

El presente documento establece el **Plan de Pruebas** del Sistema de Gestión Institucional del Albergue "Ajo'olnáj — Tu Techo". Su propósito es definir la estrategia, los escenarios y los criterios que se emplearán para verificar que el software cumple correctamente con los requerimientos funcionales establecidos en la especificación de requerimientos (SRS) organizada por fases.

La planificación de pruebas se realiza antes de la finalización del desarrollo completo del sistema, siguiendo las buenas prácticas de Ingeniería de Software, con el fin de anticipar hallazgos, reducir costos de corrección y garantizar la calidad del producto entregado.

Descripción del Proyecto

Nombre del proyecto

Sistema de Gestión Institucional — Albergue "Ajo'olnáj — Tu Techo"

Objetivo general

Desarrollar una plataforma digital que integre la gestión operativa, clínica y administrativa del albergue, sustituyendo los procesos realizados en formato papel (bitácoras, formatos clínicos, inventarios y listas de verificación) por un sistema web centralizado, seguro y basado en roles.

Problema que resuelve

El albergue atiende a personas en situación de calle en la ciudad de Campeche, México. Actualmente, los procesos de admisión, seguimiento clínico, control de estancias, inventario de medicamentos e insumos, y reportes se manejan mediante documentos de Word, Excel y formato impreso. Esto provoca:

- Pérdida o duplicación de información.
- Dificultad para dar seguimiento integral a cada residente.
- Falta de trazabilidad en movimientos de inventario.
- Ausencia de alertas automáticas para vencimiento de permanencia o medicamentos.
- Limitación en la generación de reportes para toma de decisiones.

Tecnologías utilizadas

Componente	Tecnología	Versión
Backend	PHP / Laravel	8.2.29 / 12.x
Frontend	Blade + Tailwind CSS + Alpine.js	—
Base de datos	PostgreSQL	12.17
Autenticación	Laravel Breeze	—
Assets	Vite	—
Contenedores	Docker / Laravel Sail	—
Node.js	Node.js / npm	22.17.0 / 10.9.2

Arquitectura general del sistema

El sistema utiliza una arquitectura modular basada en **Domain-Driven Design** con 11 dominios bajo **app/Domain/**. Cada dominio contiene sus propios Modelos, Servicios y Políticas de autorización. Las rutas se organizan en archivos modulares bajo **routes/modules/**. El sistema implementa un **RBAC personalizado** con 7 roles y permisos.

Módulos principales

#	Módulo	Descripción
1	Acceso y RBAC	Autenticación, 7 roles, permisos granulares, gestión de empleados y usuarios
2	Catálogos	Datos maestros (20+ catálogos), cascade geográfico, catálogos simples genéricos
3	Expedientes y Admisión	Registro de personas, expedientes con folio automático, búsqueda difusa (trigram)

4	Estancias y Camas	Gestión de estancias, periodos de permanencia (3-6 meses), asignación de camas, pertenencias
5	Bitácora	Registro diario de atención, servicios, visitas e incidencias
6	Clínico	Cédula clínica interdisciplinaria, plan individual de atención, signos vitales
7	Valoraciones	Valoraciones por área (Trabajo Social, Enfermería, Psicología)
8	Egresos y Evaluaciones	Evaluaciones trimestrales, egreso, canalizaciones, cierre de expediente
9	Inventario	Medicamentos (lotes, caducidad, administración), insumos, recepciones, requisiciones
10	Operación	Listas de verificación (checklists) e inspecciones por turno
11	Sistema	Adjuntos, auditoría, notificaciones

Estrategia General de Pruebas

Tipos de pruebas

El equipo realizará los siguientes tipos de prueba:

Tipo de prueba	Descripción	Alcance
Unitarias	Verifican métodos y funciones individuales de modelos, servicios y políticas	Lógica de negocio aislada
De integración	Verifican la comunicación correcta entre módulos (controlador-servicio-modelo, rutas-vistas)	Flujo entre dos o más componentes
End-to-End (E2E)	Simulan recorridos completos del usuario desde la interfaz	Procesos de negocio completos
De regresión	Se ejecutan después de cada cambio significativo para confirmar que funcionalidades existentes no se afectaron	Todo el sistema
De validación de datos	Verifican el manejo correcto de entradas inválidas, vacías y límites	Formularios y endpoints

Momento de ejecución

Tipo de prueba	Fase de desarrollo
Unitarias	Continua, durante el desarrollo de cada módulo (cada fase)
De integración	Al finalizar cada fase, al conectar módulos entre sí

E2E	Después de completar las fases 1-7 (módulos core operativos)
De regresión	Después de cada cambio o corrección de errores
De validación de datos	Durante el desarrollo de cada formulario y endpoint

Responsables

Tipo de prueba	Responsable
Unitarias	El desarrollador que implementó el componente
De integración	Un integrante del equipo diferente al desarrollador
E2E	Cualquier integrante del equipo (simulando el rol de usuario)
De regresión	El desarrollador que realizó el cambio
De validación de datos	El desarrollador del módulo + revisión cruzada

Herramientas para documentar y registrar resultados

Herramienta	Uso
PHPUnit / Pest (incluido en Laravel)	Ejecución automatizada de pruebas unitarias y de integración
Markdown en el repositorio	Registro manual de escenarios E2E y resultados observados
GitHub Issues	Registro de errores encontrados durante las pruebas
Bitácora de auditoría del sistema	Verificación de trazabilidad post-cambio
Navegador web (Chrome/Firefox)	Ejecución manual de pruebas E2E
Consola del navegador	Verificación de errores de JavaScript y peticiones AJAX

Justificación

La estrategia se enfoca en pruebas unitarias como primera línea de defensa, dado que el sistema maneja lógica de negocio compleja (cálculo de permanencia, control de stock por lotes, RBAC por roles). Las pruebas de integración son críticas porque los 11 dominios interactúan frecuentemente. Las E2E validan los flujos reales que realizarán los usuarios del albergue (personal de trabajo social, enfermería, psicología, cocina y dirección).

Plan de Pruebas Unitarias

PU-01: Cálculo de fecha de egreso prevista

Campo	Descripción
Identificador	PU-01
Módulo	Estancias
Función o método a probar	Estancia::calcularFechaEgresoPrevista() o lógica equivalente en el servicio de estancias
Objetivo de la prueba	Verificar que al crear una estancia, la fecha de egreso prevista se calcule correctamente sumando 3 meses a la fecha de ingreso
Datos de entrada	Fecha de ingreso: 2026-01-15
Resultado esperado	Fecha de egreso prevista: 2026-04-15 (exactamente 3 meses después)
Resultado obtenido	Pendiente
Estado	Pendiente

PU-02: Generación automática de folio de expediente

Campo	Descripción
Identificador	PU-02
Módulo	Expedientes
Función o método a probar	Expediente::generarFolio() o servicio de expedientes

Objetivo de la prueba	Verificar que el folio se genere con el formato correcto: VIS-AAAA-NNNNN para visitantes o RES-AAAA-NNNNN para residentes, con secuencia incremental
Datos de entrada	Tipo de persona: "Residente", Año actual: 2026, Último folio existente: RES-2026-00003
Resultado esperado	Nuevo folio generado: RES-2026-00004
Resultado obtenido	Pendiente
Estado	Pendiente

PU-03: Verificación de permisos por rol (RBAC)

Campo	Descripción
Identificador	PU-03
Módulo	Acceso
Función o método a probar	Role y Permission models — Gates/Policiés del sistema RBAC
Objetivo de la prueba	Verificar que un usuario con rol direccion (solo lectura) no pueda ejecutar acciones de escritura (crear, editar, eliminar) en ningún módulo
Datos de entrada	Usuario con rol direccion , intento de acceder a POST /empleados
Resultado esperado	Respuesta HTTP 403 (Forbidden) — acceso denegado
Resultado obtenido	Pendiente
Estado	Pendiente

PU-04: Cálculo de stock disponible de medicamento por lotes

Campo	Descripción
--------------	--------------------

Identificador	PU-04
Módulo	Inventario
Función o método a probar	Medicamento::stockDisponible() o servicio de medicamentos — suma de cantidades de todos los lotes activos con stock > 0
Objetivo de la prueba	Verificar que el stock total de un medicamento se calcule sumando las cantidades disponibles de todos sus lotes (considerando movimientos de entrada y salida)
Datos de entrada	Medicamento "Paracetamol 500mg": Lote A (100 unidades), Lote B (50 unidades), movimiento de salida de 30 unidades del Lote A
Resultado esperado	Stock disponible: 120 unidades (70 del Lote A + 50 del Lote B)
Resultado obtenido	Pendiente
Estado	Pendiente

PU-05: Validación de límite de permanencia (6 meses máximo)

Campo	Descripción
Identificador	PU-05
Módulo	Estancias
Función o método a probar	Servicio de estancias — validación al intentar extender una estancia
Objetivo de la prueba	Verificar que el sistema impida extender una estancia cuando ya se ha alcanzado el límite máximo de 6 meses de permanencia

Datos de entrada	Estancia con fecha de ingreso 2026-01-01 , fecha de egreso prevista actual 2026-06-30 (6 meses), intento de extensión
Resultado esperado	El sistema rechaza la extensión y muestra mensaje: "La estancia ha alcanzado el límite máximo de permanencia (6 meses)"
Resultado obtenido	Pendiente
Estado	Pendiente

PU-06: Validación de administración de medicamento con stock insuficiente

Campo	Descripción
Identificador	PU-06
Módulo	Inventario
Función o método a probar	Servicio de administración de medicamentos
Objetivo de la prueba	Verificar que el sistema impida administrar un medicamento cuando el stock disponible es menor a la cantidad solicitada
Datos de entrada	Medicamento con stock disponible: 5 unidades, cantidad a administrar: 10 unidades
Resultado esperado	El sistema rechaza la administración y retorna error de stock insuficiente
Resultado obtenido	Pendiente
Estado	Pendiente

PU-07: Búsqueda difusa de personas por trigram (prevención de duplicados)

Campo	Descripción
--------------	--------------------

Identificador	PU-07
Módulo	Expedientes
Función o método a probar	Endpoint GET /expedientes/api/buscar-personas con parámetro de búsqueda
Objetivo de la prueba	Verificar que la búsqueda difusa con trigram encuentre personas con nombres similares (tolerancia a errores de escritura)
Datos de entrada	Nombre registrado: "María Elena López", término de búsqueda: "Maria Lopez"
Resultado esperado	La búsqueda retorna el registro de "María Elena López" como resultado coincidente
Resultado obtenido	Pendiente
Estado	Pendiente

Plan de Pruebas de Integración

PI-01: Inicio de sesión → Acceso al Dashboard

Campo	Descripción
Identificador	PI-01
Módulos involucrados	Autenticación (Breeze) → RBAC → Dashboard
Objetivo de la prueba	Verificar que un usuario autenticado con credenciales correctas acceda al dashboard y que el contenido del panel se adapte a su rol asignado
Flujo esperado	1. El usuario ingresa email y contraseña en /login . 2. Laravel Breeze valida las credenciales. 3. El sistema verifica que la cuenta esté activa (activo = true). 4. Se redirige

	a /dashboard . 5. El dashboard muestra menú y módulos según el rol del usuario
Resultado esperado	El usuario accede al dashboard y visualiza únicamente los módulos correspondientes a su rol. Un administrador ve todos los módulos; un trabajador social ve expedientes, estancias y egresos
Estado	Pendiente

Justificación: Esta integración es crítica porque es el punto de entrada principal al sistema. Un fallo aquí bloquea completamente el acceso. Además, la personalización del menú según el rol valida que el RBAC funcione correctamente desde el inicio.

PI-02: Registro de empleado → Creación de usuario → Asignación de rol

Campo	Descripción
Identificador	PI-02
Módulos involucrados	Empleados (Acceso) → Usuarios (Breeze) → Roles (RBAC)
Objetivo de la prueba	Verificar que al crear un empleado, se genere su cuenta de usuario, se le asigne el rol correcto y pueda iniciar sesión con las credenciales generadas
Flujo esperado	1. Admin crea un empleado con datos personales y área. 2. El sistema genera la cuenta de usuario (email, contraseña temporal). 3. Se asigna el rol correspondiente. 4. El nuevo usuario inicia sesión. 5. El sistema carga los permisos del rol

	asignado
Resultado esperado	El empleado queda registrado, su cuenta de usuario está activa, el rol está asignado correctamente y puede acceder a los módulos que le corresponden
Estado	Pendiente

Justificación: La creación de empleados es una operación transaccional compleja que involucra empleado + documentos obligatorios + usuario + rol. Si falla esta cadena, el sistema no puede incorporar nuevo personal operativo.

PI-03: Apertura de expediente → Creación de estancia → Asignación de cama

Campo	Descripción
Identificador	PI-03
Módulos involucrados	Personas → Expedientes → Estancias → Camas
Objetivo de la prueba	Verificar el flujo completo de admisión: desde el registro de una persona hasta la asignación de una cama en un dormitorio
Flujo esperado	1. Se registra una persona usuaria (con datos personales). 2. Se crea su expediente (folio automático). 3. Se abre una estancia asociada al expediente. 4. Se asigna una cama disponible en un dormitorio. 5. La cama aparece como ocupada en el tablero de camas

Resultado esperado	La persona queda registrada con su expediente, estancia activa y cama asignada. El tablero de camas refleja la ocupación correctamente
Estado	Pendiente

Justificación: Este es el flujo principal del albergue: admitir una persona y ubicarla. La integración entre estos cuatro módulos es esencial para la operación diaria. Un fallo aquí impediría el ingreso de nuevos residentes.

PI-04: Estancia activa → Cédula clínica → Valoraciones → Egreso

Campo	Descripción
Identificador	PI-04
Módulos involucrados	Estancias → Clínico → Valoraciones → Egresos
Objetivo de la prueba	Verificar que el ciclo de vida clínico de una estancia funcione correctamente: desde la evaluación interdisciplinaria inicial hasta el egreso
Flujo esperado	1. Con una estancia activa, Trabajo Social realiza la cédula clínica. 2. Enfermería registra signos vitales. 3. Psicología registra su valoración. 4. Se autoriza el plan individual. 5. Se realiza la evaluación trimestral (requiere los 3 dictámenes). 6. Trabajo Social genera el acta de egreso. 7. Se cierra el expediente
Resultado esperado	Todos los registros clínicos quedan asociados a la estancia, la evaluación trimestral se completa con los 3 dictámenes, el egreso se registra correctamente y el expediente se marca como cerrado

Estado	Pendiente
---------------	-----------

Justificación: Este flujo representa el ciclo de vida completo de un residente en el albergue. La integración entre módulos clínicos y de egreso es vital para garantizar la continuidad de la atención y el cumplimiento de los protocolos institucionales.

PI-05: Inventario de medicamentos → Administración a residente → Bitácora

Campo	Descripción
Identificador	PI-05
Módulos involucrados	Inventario (Medicamentos) → Estancias → Bitácora
Objetivo de la prueba	Verificar que al administrar un medicamento a un residente, se descuenta del stock y quede registrado en la bitácora de atención del día
Flujo esperado	1. Enfermería selecciona un medicamento con stock disponible. 2. Registra la administración a un residente específico. 3. El sistema descuenta la cantidad del lote correspondiente. 4. Se crea un registro en la bitácora de atención del día
Resultado esperado	El stock del medicamento se reduce, la administración queda registrada con residente, medicamento, cantidad y fecha, y aparece en la bitácora diaria
Estado	Pendiente

Justificación: La gestión de medicamentos es un proceso crítico de seguridad. Un fallo en la integración podría provocar doble administración, stock negativo o falta de trazabilidad farmacológica.

Plan de Pruebas End-to-End (E2E)

E2E-01: Admisión completa de un nuevo residente

Campo	Descripción
Identificador	E2E-01
Nombre del escenario	Admisión completa de un nuevo residente
Actor principal	Trabajo Social
Precondiciones	El usuario tiene sesión activa con rol trabajo_social . Existen dormitorios con camas disponibles. Los catálogos de motivo de ingreso, discapacidad y adicción están poblados
Pasos a seguir	1. Iniciar sesión con credenciales de Trabajo Social. 2. Navegar a Expedientes > Nuevo Expediente. 3. Buscar la persona por nombre (verificar que no exista previamente). 4. Completar el formulario de registro: nombre, fecha de nacimiento, CURP (opcional), sexo, estado civil, escolaridad, dirección, motivo de ingreso, discapacidades, adicciones, situación de vulnerabilidad. 5. Guardar el expediente y verificar el folio generado. 6. Crear una nueva estancia asociada al expediente. 7. Navegar al tablero de camas. 8. Asignar una cama disponible en un dormitorio Transitorio. 9. Verificar que la cama aparezca como ocupada. 10. Registrar pertenencias del residente. 11. Cerrar sesión

Resultado esperado	La persona queda registrada con folio RES-2026-XXXXX , tiene una estancia activa con cama asignada, sus pertenencias quedan registradas y el tablero de camas refleja la ocupación
Estado	Pendiente

E2E-02: Proceso clínico interdisciplinario y evaluación trimestral

Campo	Descripción
Identificador	E2E-02
Nombre del escenario	Evaluación clínica interdisciplinaria y evaluación trimestral
Actor principal	Trabajo Social, Enfermería, Psicología (tres usuarios diferentes)
Precondiciones	Existe un residente con estancia activa y cama asignada. Los tres usuarios (trabajo social, enfermería, psicología) tienen credenciales válidas
Pasos a seguir	1. Trabajo Social inicia sesión y registra la cédula clínica inicial (área de Trabajo Social). 2. Trabajo Social crea el plan individual de atención. 3. Enfermería inicia sesión, registra signos vitales del residente (presión arterial, frecuencia cardíaca, temperatura, saturación de oxígeno). 4. Enfermería realiza su valoración de enfermería. 5. Psicología inicia sesión y realiza su valoración psicológica. 6. Se autoriza el plan individual (cada área autoriza su sección). 7. Trabajo Social crea la

	evaluación trimestral con los tres dictámenes. 8. Se verifica que la evaluación se complete exitosamente (requiere los 3 dictámenes)
Resultado esperado	La cédula clínica, signos vitales, valoraciones de las tres áreas y la evaluación trimestral quedan registrados y asociados a la estancia. El plan individual muestra las tres columnas correctamente. La evaluación trimestral se completa con los tres dictámenes
Estado	Pendiente

E2E-03: Egreso de residente con cierre de expediente

Campo	Descripción
Identificador	E2E-03
Nombre del escenario	Egreso de residente y cierre de expediente
Actor principal	Trabajo Social
Precondiciones	Existe un residente con estancia activa, al menos una evaluación trimestral completada y el plazo de permanencia está próximo a vencer o se requiere egreso anticipado

Pasos a seguir	1. Trabajo Social inicia sesión. 2. Navega al expediente del residente. 3. Revisa el historial de estancias y evaluaciones. 4. Registra el informe final interdisciplinario. 5. Genera el acta de egreso (solo Trabajo Social puede crear egresos). 6. Registra la causa de egreso y opcionalmente una canalización a institución externa. 7. Entrega las pertenencias pendientes del residente. 8. Cierra la estancia. 9. Verifica que el tablero de camas libere la cama. 10. Cierra el expediente. 11. Verifica que el expediente aparezca como cerrado en la lista
Resultado esperado	El acta de egreso queda registrada con firma y documentos adjuntos, la cama se libera en el tablero, el expediente se marca como cerrado y el residente ya no aparece como activo
Estado	Pendiente

E2E-04: Registro de bitácora diaria con servicios e incidencia

Campo	Descripción
Identificador	E2E-04
Nombre del escenario	Registro completo de bitácora diaria
Actor principal	Encargado de turno
Precondiciones	Existen residentes activos con cama asignada. Hay turnos Matutino y Nocturno configurados. El comedor

	y los acompañamientos están registrados
Pasos a seguir	1. Iniciar sesión con rol encargado_turno. 2. Navegar a Bitácora > Nueva entrada. 3. Seleccionar el turno actual. 4. Seleccionar residentes que consumieron servicios (desayuno, comida, cena). 5. Marcar los acompañamientos correspondientes. 6. Seleccionar el comedor utilizado. 7. Registrar una visita recibida por un residente (datos del visitante, hora). 8. Registrar una incidencia (tipo, descripción, residente involucrado). 9. Guardar la entrada de bitácora. 10. Verificar que los registros aparezcan en la lista de bitácora del día
Resultado esperado	La bitácora del día queda registrada con todos los servicios prestados, la visita y la incidencia. Los datos son visibles en la consulta de bitácora
Estado	Pendiente

Matriz de Cobertura de Pruebas

Requerimiento	Descripción	Prueba Unitaria	Integración	E2E
RF-01	Autenticación y control de acceso por roles	PU-03	PI-01	E2E-01
RF-02	Gestión de catálogos y datos maestros	—	—	—

RF-03	Gestió n de emple ados y creaci ón de usuari os	—	PI-02	—
RF-04	Regis tro de perso nas y apert ura de exped ientes	PU-02, PU-07	PI-03	E2E-01
RF-05	Gestió n de estanci as y asigna ción de camas	PU-01, PU-05	PI-03	E2E-01
RF-06	Bitácora diaria, visitas e incidenc ias	—	—	E2E-04
RF-07	Módulo clínico: cédula, plan, signos vitales	—	PI-04	E2E-02

RF-08	Valoraciones por área (Trabajo Social, Enfermería, Psicología)	—	PI-04	E2E-02
RF-09	Evaluaciones trimestrales y egreso	—	PI-04	E2E-03
RF-10	Inventario de medicamentos y administración	PU-04, PU-06	PI-05	—
RF-11	Inventario de insumos, recepciones y requisiciones	—	—	—
RF-12	Listas de verificación e inspecciones por turno	—	—	—
RF-13	Sistema de notificaciones y alertas	—	—	—

RF-14	Auditoría y trazabilidad	—	—	—
RF-15	Adjuntos y documentos	—	—	—

Criterios de Aceptación

Una funcionalidad será considerada **aprobada** cuando cumpla con todos los siguientes criterios:

Funcionamiento correcto

- La funcionalidad ejecuta la operación solicitada sin errores de ejecución.
- Los datos se procesan y almacenan correctamente en la base de datos.
- La respuesta del sistema es consistente con la acción realizada.

Cumplimiento del requerimiento

- La funcionalidad implementa el comportamiento descrito en el requerimiento funcional correspondiente (SRS por fases).
- No existen desviaciones no autorizadas respecto a la especificación.

Manejo adecuado de errores

- Los errores de aplicación se manejan con mensajes claros y comprensibles para el usuario.
- No se expone información técnica sensible al usuario final.
- Las excepciones se registran en el log de la aplicación.
- El sistema no cae en estado inconsistente ante errores.

Validación de datos

- Los campos obligatorios se validan en el lado del cliente (JavaScript/Alpine.js) y del servidor (Laravel Request/Validation).
- Los datos inválidos son rechazados con mensajes descriptivos.
- Se respetan las restricciones de tipo, longitud y formato de cada campo.
- Las validaciones únicas (como CURP) se verifican correctamente.

Integración correcta con otros módulos

- Las operaciones que involucran múltiples módulos se ejecutan en una transacción consistente.
- Los cambios en un módulo se reflejan correctamente en los módulos dependientes.
- No se producen duplicidades ni pérdida de datos al cruzar módulos.

Ausencia de errores críticos

- No existen errores que impidan el uso de la funcionalidad.
- No hay errores de seguridad (inyección SQL, XSS, CSRF).
- No se producen errores de base de datos (violaciones de integridad, claves foráneas).
- No hay errores de lógica que produzcan datos inconsistentes.

Control de acceso

- Solo los usuarios con el rol autorizado pueden ejecutar la acción.
- Los usuarios sin permiso reciben respuesta HTTP 403.
- Las operaciones exclusivas de un rol no son accesibles desde otros roles.

Riesgos y Limitaciones

#	Riesgo / Limitación	Probabilidad	Impacto	Acción de mitigación
1	Falta de datos de prueba realistas — Los catálogos y registros de prueba pueden no reflejar la diversidad de casos reales del albergue	Alta	Medio	Utilizar Faker para generar datos variados; crear seeders con escenarios representativos (múltiples tipos de discapacidad, edades, motivos de ingreso)

2	Dependencia de servicios externos — El sistema actualmente no integra servicios externos, pero futuras canalizaciones podrían requerir APIs de instituciones gubernamentales	Baja	Alto	Diseñar la integración con interfaces abstractas; crear mocks para servicios externos en pruebas; documentar las dependencias
3	Integraciones aún no desarrolladas — La Fase 8 (Inventario/Operación) y Fase 9 (Vista Móvil) están en progreso o pendientes	Alta	Alto	Priorizar pruebas de integración en los módulos completados (Fases 1-7); diferir pruebas E2E de módulos incompletos; usar datos mock para módulos

				pendientes
4	Cambios frecuentes en los requerimientos — Los requerimientos pueden ajustarse durante el desarrollo según retroalimentación del albergue	Media	Alto	Mantener el SRS como documento vivo versionado en Git; documentar cada cambio y su impacto en pruebas existentes; ejecutar pruebas de regresión después de cada cambio
5	Tiempo limitado para realizar pruebas — El cronograma académico impone	Alta	Alto	Priorizar pruebas en los módulos críticos (admisión, clínico, egreso); automatizar pruebas

	fechas de entrega que limitan el alcance de las pruebas			unitarias con PHPUnit; documentar escenarios E2E prioritarios
6	Entorno de desarrollo con Docker/WSL — El rendimiento del entorno Docker montado desde Windows puede afectar la ejecución de pruebas automatizadas	Media	Medio	Ejecutar el proyecto desde Ubuntu/WSL para mejor rendimiento; usar bases de datos de prueba separadas del entorno de desarrollo
7	Conocimiento limitado de herramientas de testing — El equipo puede tener poca experiencia con PHPUnit o	Media	Medio	Consultar la documentación oficial de Laravel Testing; empezar con pruebas unitarias simples y escalar complejidad

	Pest			; revisar ejemplos en el directorio tests/ del proyecto
8	Base de datos compartida entre desarrollo y pruebas — Las pruebas podrían afectar los datos de desarrollo si no se usa una base separada	Media	Alto	Configurar una base de datos de prueba independiente en phpunit.xml I; usar transacciones reversibles en pruebas de integración; nunca ejecutar pruebas contra la base de producción

Conclusión

La planificación de pruebas antes de la finalización del desarrollo del Sistema de Gestión Institucional del Albergue "Ajo'olnaj — Tu Techo" es una práctica fundamental para garantizar la calidad y confiabilidad del producto. Al definir anticipadamente los escenarios de prueba, los criterios de aceptación y la estrategia a seguir, el equipo establece un marco de referencia claro que permite:

Importancia de planificar antes de desarrollar completamente: Planificar las pruebas desde etapas tempranas permite detectar inconsistencias en los requerimientos, identificar dependencias entre módulos y anticipar escenarios de fallo que podrían ser costosos de corregir si se descubren en fases avanzadas. En un sistema con 11 dominios modulares y 7 roles de acceso, la complejidad de interacciones hace que las pruebas no planificadas sean insuficientes para cubrir todos los escenarios relevantes.

Beneficios de una estrategia bien definida: Una estrategia de pruebas estructurada aporta visibilidad sobre la cobertura real del sistema, permite priorizar el esfuerzo de prueba en los módulos más críticos (admisión, clínico, egreso, inventario) y facilita la detección temprana de defectos. La automatización de pruebas unitarias con PHPUnit, combinada con escenarios E2E documentados, equilibra velocidad de ejecución con exhaustividad de cobertura.

Contribución a la calidad del proyecto: Este documento servirá como guía operativa durante las fases finales de desarrollo (Fase 8 y 9) y como evidencia de que el equipo sigue un proceso disciplinado de aseguramiento de calidad. Los criterios de aceptación definidos proporcionan estándares objetivos e medibles, mientras que la matriz de cobertura garantiza que todos los requerimientos funcionales cuenten con al menos una prueba asociada. La identificación de riesgos y sus mitigaciones permite al equipo tomar decisiones informadas cuando las limitaciones de tiempo o recursos pongan a prueba la planificación.

En síntesis, este Plan de Pruebas fortalece el desarrollo del sistema al incorporar la verificación como parte integral del proceso, no como una actividad posterior, contribuyendo a la entrega de un producto confiable, seguro y funcional para la comunidad del Albergue Ajo'olnaj.

